라즈베리파이 카메라로 사람 추적하는 알고리즘 (open CV와 media pipe 이용)

```
import cv2                          // opencv 라이브러리 불러오기.
import mediapipe as mp              // mediapipe 라이브러리 불러오기.

mp_pose = mp.solutions.pose         // mediapipe 라이브러리 안에 있는 pose 모듈 저장.

pose = mp_pose.Pose(model_complexity=0, min_detection_confidence=0.5,
                    min_tracking_confidence=0.5)
```

Note. mediapipe 라이브러리

- mediapipe 라이브러리
  - Pose module
  - Hands module
  - Face module
  - Holistic module
  - ⋮

// pose 모듈 생성

- model_complexity=0 (pose 모듈의 무게 설정) (0 : 가벼움)
- min_detection_confidence (신뢰도) 사람인식
- min_tracking_confidence (사람추적 신뢰도) (정확도)

```
mp_draw = mp.solutions.drawing_utils.   // pose 모듈의 결과를 시각화하기 위한 도구를 가져온다.

cap = cv2.VideoCapture(0, cv2.CAP_V4L2)   // cv2.VideoCapture 이라는 클래스에서 (0, cv2.CAP_V4L2) 라는 인자를 대입한 객체를 cap 변수에 저장.
```

- cv2.VideoCapture: Open CV에서 영상파일을 연다
- 0: 첫번째 카메라
- cv2.CAP_V4L2: 카메라 인식 안정

```
while cap.isOpened():                   // 카메라가 정상적으로 열려있는 동안. (계속실행)
    ret, frame = cap.read()            // 프레임을 읽고,
    if not ret:                        // 프레임을 못 읽었다면
        break                          // while문 종료.
    frame = cv2.resize(frame, (640, 480))   // 성능을 위해 프레임 해상도 640 x 480 으로 고정.
    img_rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
    results = pose.process(img_rgb)    // Mediapipe pose 모델 실행 후 결과 저장.
```

- ret: 성공여부 저장 (True/False)
- frame: 실제 프레임 저장

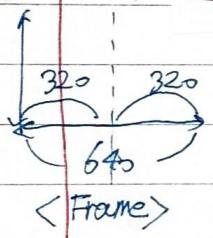
morning glory

// 결과 (results) :
- results.pose_landmarks = 33개 관절 좌표
- results.pose_world_landmarks = 3D 관절 좌표
- None = 아무것도 없음

```
if results.pose_landmarks :                    // 사람이 인식되면
    mp_draw.draw_landmarks(frame, results.pose_landmarks,
                           mp_pose.POSE_CONNECTIONS)
                                   // 화면에 인식된 사람의 뼈대를 그린다
    lm = results.pose_landmarks.landmark   // mediapipe pose 모듈이 인식한 사람관절 가져옴
    h, w, _ = frame.shape   → 현재 영상 프레임의 크기
    cx = int((lm[11].x + lm[12].x) * w/2)   → 어깨 중심의 x좌표
    cy = int((lm[11].x + lm[12].y) * h/2)   → 어깨 중심의 y좌표
    cv2.circle(frame, (cx, cy), 10, (0, 255, 0), -1)   // 사람의 중심좌표에 초록점 표시
    error = cx - 320   → 사람중심 좌표가 얼만큼(좌/우로) 치우쳐 졌는지 계산
    print("중심 좌표 : {cx}, 오차 : {error}")
        실시간 좌표출력   실시간 error 출력
```

11, 12 번 = 왼쪽, 오른쪽 어깨 좌표

사람 중심점 (cx, cy)




```
    cv2.imshow("Atov Robot Version", frame)   → 최종 화면 출력

    if cv2.waitKey(1) & 0xFF == ord('q'):   → q를 입력하면 (while문 종료 조건)
        break   → while 문 종료

cap.release()   → 카메라 사용종료
cv2.destroyAllWindows()   → 모든 창 닫기
```

원의 중심 | 원의 반지름 | 초록색 | 채워진 원